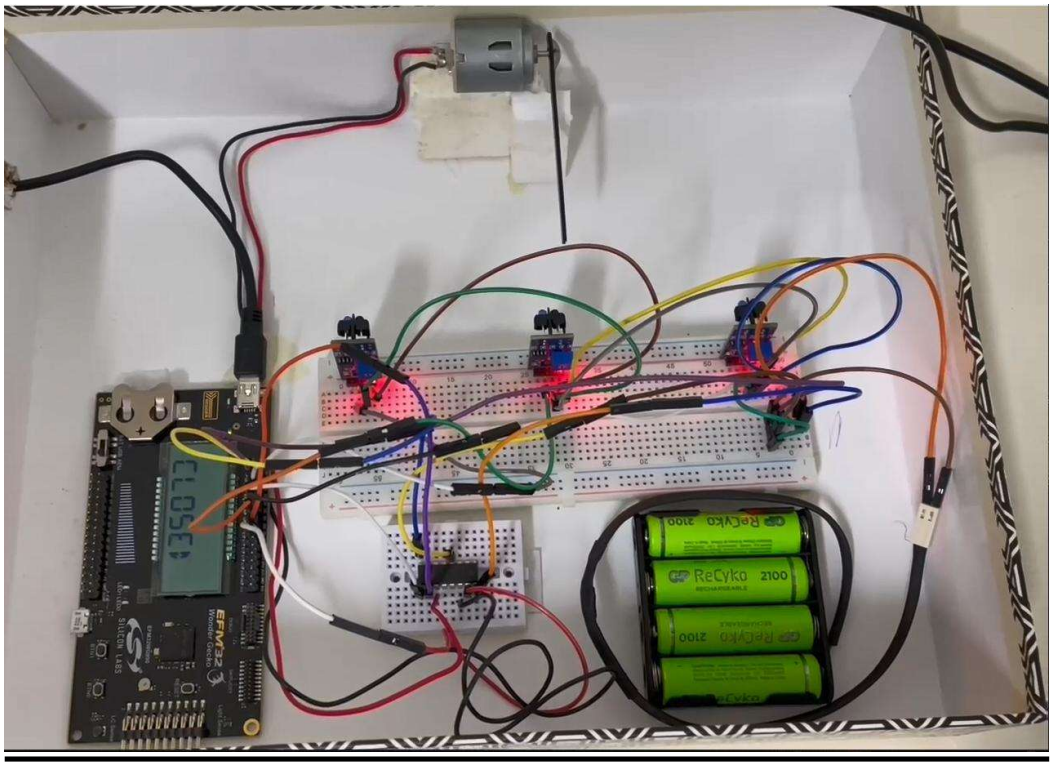
מטרת הפרויקט:

בקרת שער של חניון שנפתח ע"י זיהוי רכב באמצעות חיישני TCRT5000 IR, יכולת לדעת אם הנהג נכנס או יוצא או נמצא באמצע . ומסך LCD יציג אם השער פתוח או סגור.

תמונה לפרויקט:



סרטון להפעלת המערכת והדגמה לפרויקט:

https://youtu.be/fC0qOlvIKPs

אופן פעולת המערכת:

בחיבור המערכת לחשמל, המערכת תתחיל לעבוד מייד, כלומר החיישנים יתחילו לזהות רכב נכנס ורכב יוצא.

חיישן A ו חיישן C הם החיישנים המזהים אם רכב נכנס ואם רכב יוצא, וחיישן B הוא נמצה באמצע כדי שיהיה safety mechanism במערכת שלנו.

בהתחלה, אם השער סגור וחיישן A או C מזהה רכב, אז השער נפתח ומסך הLCD יציג "open" , וכאשר הרכב עובר את החיישן בצד השני של השער אז השער נסגר ומסך הLCD יציג "closed".

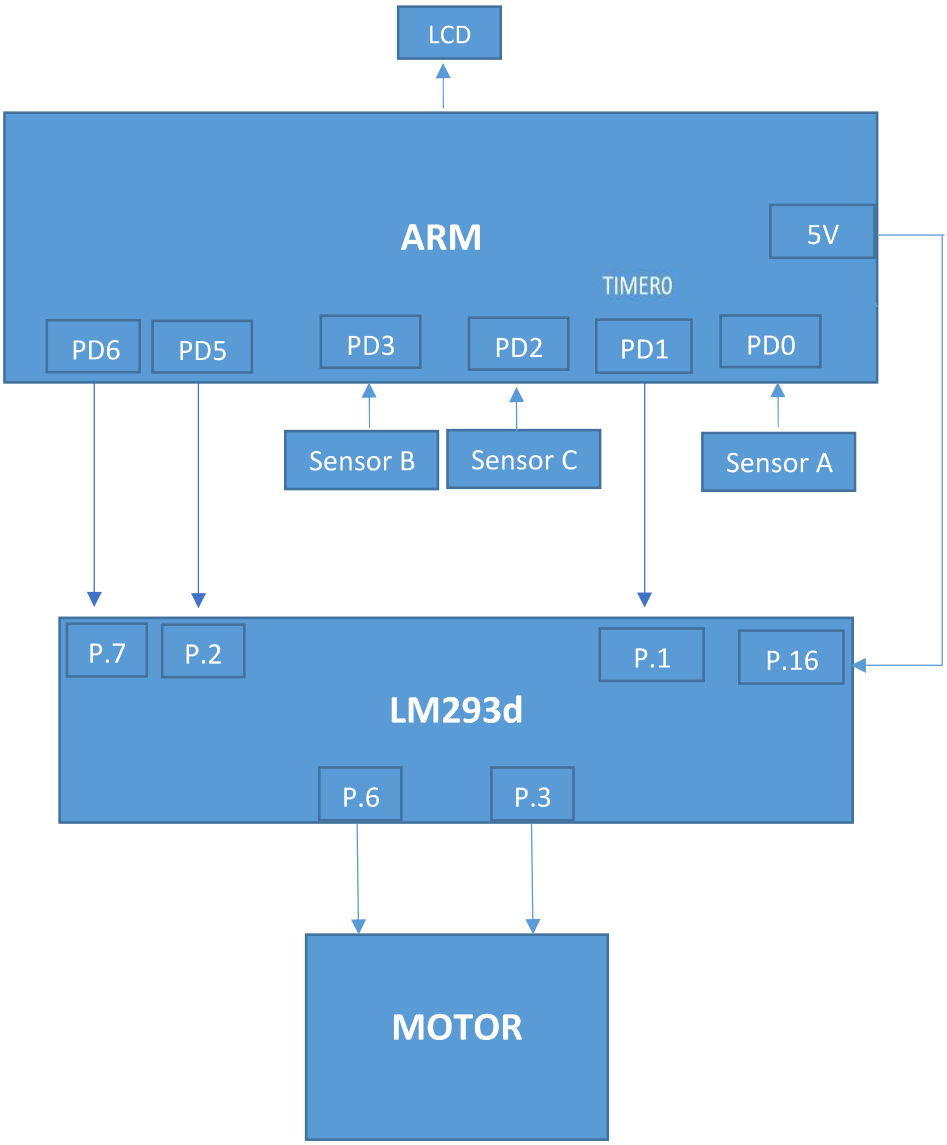
במידה וחיישן B מזהה משהו כאשר השער פתוח ,אז השער לא ייסגר עד שהגוף שחוסם את חיישן B זז.

מצד שני אם השער סגור וחיישן A או C מזהה רכב, אז השער נפתח ומסך הLCD יציג "open" ואם החיישן בצד שני לא מצליח לזהות את הרכב אז השער יסגור באופן אוטומטי אחרי 5 שניות ומסך הLCD יציג "closed". (כמובן אם חיישן B מזהה משהו כאשר השער פתוח ,אז השער לא ייסגר עד שהגוף שחוסם את חיישן B זז).

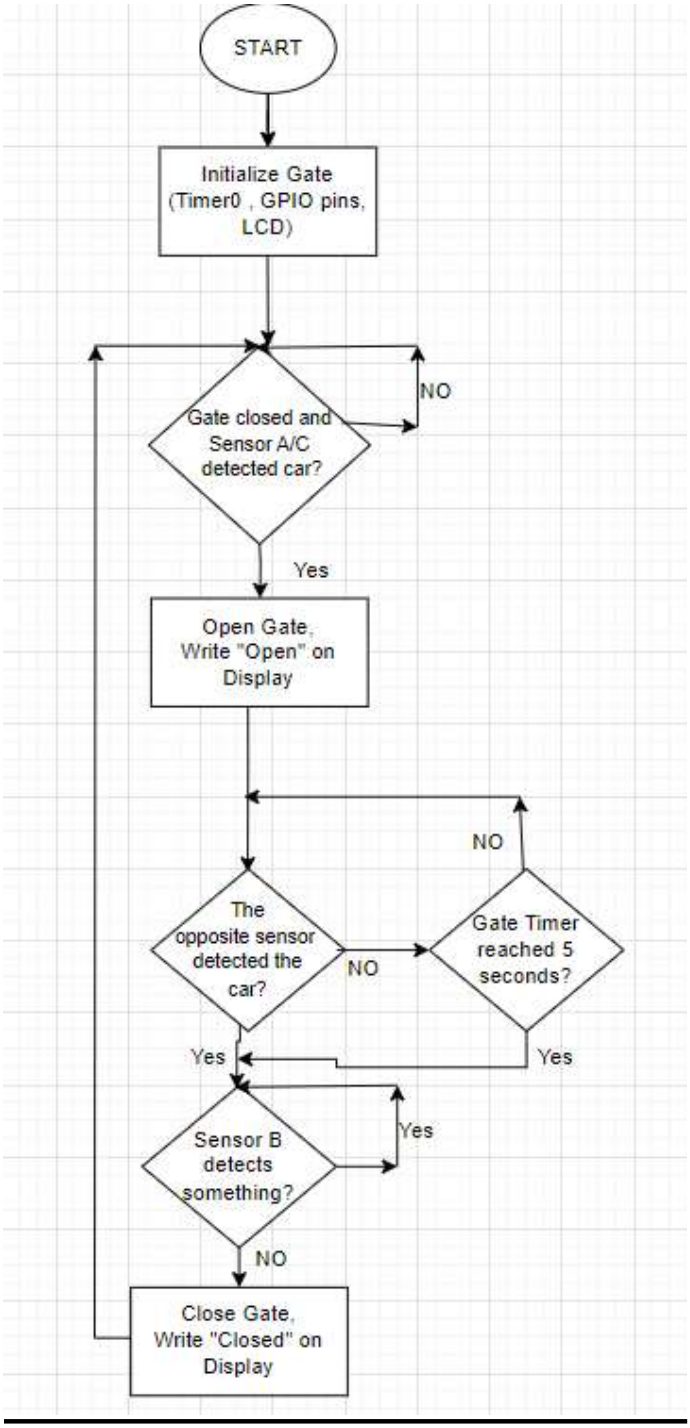
אופן פעולת IR SENSOR (TCRT5000)

- החיישן משדר גל IR שפוגע בגוף וחוזר ואז החיישן קולט את הגל ומחשב מרחק.
- דגם החיישן הזה יש לו יציאה אנלוגית , וגם יציאה דיגיטלית.
- בפרויקט שלנו השתמשנו ביציאה הדיגיטלית.

תרשים מלבינים:



תרשימי זרימה:



פונקציות בקוד:

1) **Init_Device:** אתחול הבקר (gpio , timer0).
2) **Main:** הפונקציה הראשית בקוד שמעבדת נתונים מהחיישנים ומחליטה מתי השער ייפתח וייסגר.
3) **setupPWM:** ליצור את ה-PWM הנדרש בהתאם לערכים הנתונים בשביל לשנות מהירות המנוע.
4) **DELAY:** יצור השהיית זמן.
5) **runMotor:** פונקציה להפעלת המנוע.
6) **stopMotor:** פונקציה לעצירת המנוע.
7) **openDoor:** פונקציה לפתיחת השער.
8) **closeDoor:** פונקציה לסגירת השער.
9) **SysTick_Init:** פונקציה לשימוש ב Timer שמובנה ב Cortex M4.

חישוב ערכים לרכיבים:

1) **חישוב ערך הרגיסטר TOP של Timer 0 ב-ARM בשביל הפעלת מנוע DC בעזרת PWM:**

$$f_{PWM_{up}} = \frac{f_{HFPERCLK}}{2^{PRESC} * (TOP + 1)}$$

$$1000 = \frac{48_{x10^6}}{2^6 * (TOP + 1)}$$

$$TOP = 750 = 2ED_{(H)}$$

2) **חישוב ערך ה-duty cycle לרגיסטר ה-CC של Timer 0:**

$$DC_{value} = TOP * (the\ required\ duty\ cycle)$$

בפרויקט שלנו דרשנו בסגירת השער וגם בפתיחת השער :
Duty cycle = 35%

$$DC_{value} = 750 * 35\% = 270$$

צילומי מסך של הקוד:

*FULLCODE.c

```c
#include "em_device.h"
#include "em_gpio.h"
#include "em_timer.h"
#include "em_cmu.h"
#include "em_chip.h"
#include "segmentlcd.h"

#define SENSOR_A   gpioPortD,0
#define SENSOR_B   gpioPortD,3
#define SENSOR_C   gpioPortD,2
#define MOTOR_EN   gpioPortD,1
#define MOTOR_INPUT1 gpioPortD,5
#define MOTOR_INPUT2 gpioPortD,6

// Define states for door logic
#define DOOR_OPEN_A   1
#define DOOR_OPEN_C   2
#define DOOR_CLOSED 0
// Variable to keep track of the door state
volatile int doorState = DOOR_CLOSED;
// Timer to count 1ms time ticks
volatile uint32_t msTicks = 0;
// Function USED
void stopMotor(void);
void SysTick_Init(void);
void delay(uint32_t dlyTicks);
void setupPWM(void);
void Device_Init(void);
void openDoor(void);
void closeDoor(void);
void runMotor(int duration, int opening);
// SysTick Handler
void SysTick_Handler(void) {
  msTicks++; // increment counter necessary in delay()
}
// Initialize SysTick with busy wait running at bus clock
void SysTick_Init(void) {

  if (SysTick_Config(SystemCoreClock / 1000)) { // 1ms ticks
    while (1);
  }
}
void delay(uint32_t dlyTicks) {
  uint32_t curTicks = msTicks;
  while ((msTicks - curTicks) < dlyTicks);
}
```

```c
int main(void) {
  SystemInit();
  CHIP_Init();
  SysTick_Init(); // Initialize SysTick
  Init_Device();

  unsigned int lastTimeDoorOpened = 0; // Store the last time the door was opened
  bool isClosing = false; // Flag to indicate if the door is in the process of closing

  while (1) {
    // Wait for a sensor to trigger door opening
    while(doorState == DOOR_CLOSED) {
      if(GPIO_PinInGet(SENSOR_A) == 0) {
        openDoor();
        doorState = DOOR_OPEN_A;
        lastTimeDoorOpened = msTicks;
        break; // Break out of while-loop once the door is opened
      }
      if(GPIO_PinInGet(SENSOR_C) == 0) {
        openDoor();
        doorState = DOOR_OPEN_C;
        lastTimeDoorOpened = msTicks;
        break; // Break out of while-loop once the door is opened
      }
    }

    // Wait for either sensor A or C to trigger again or for 7 seconds to pass for auto close
    while(doorState != DOOR_CLOSED && !isClosing) {
      if(doorState == DOOR_OPEN_A && ((GPIO_PinInGet(SENSOR_C) == 0) || (msTicks - lastTimeDoorOpened > 7000))) {
        isClosing = true; // Set the flag to close door
      }
      if(doorState == DOOR_OPEN_C && ((GPIO_PinInGet(SENSOR_A) == 0) || (msTicks - lastTimeDoorOpened > 7000))) {
        isClosing = true; // Set the flag to close door
      }
      if(GPIO_PinInGet(SENSOR_B) == 0) { // If sensor B detects a person, do not close the door
        isClosing = false;
        lastTimeDoorOpened = msTicks; // Reset the timer to give more time
      }
    }

    // Execute door closing procedure
    while(isClosing) {
      closeDoor();
      doorState = DOOR_CLOSED;
      isClosing = false; // Reset closing flag
    }
  }
}
```

```c
void setupPWM(void) {
    int presc = 6;
    //freq_hfprerclk = 48000000;
    //freq_pwm=1000;
    CMU_ClockEnable(cmuClock_TIMER0, true);
    GPIO_PinModeSet(MOTOR_EN, gpioModePushPullDrive, 0);
  //TIMER0 Init
    TIMER0->CTRL = (presc<<24);
    TIMER0->CNT = 0x0;  // clear TIMER0 counter value
    TIMER0->TOP = 750;   // Set TOP value, 64 = 2^presc
    TIMER0->CC->CCV =270;  // set compare value
    TIMER0->ROUTE = (3<<16)|0x1;// conect channel to port D pin 1
    TIMER0->CC->CTRL = 3;// config PWM
    TIMER0->CMD = 1; // start timer
}
void openDoor(void) {
    runMotor(350, 1); // Run motor for 350 micro-seconds to open
    SegmentLCD_Write("OPENING");
    delay(2000);
    SegmentLCD_Write("OPEN");
}
void closeDoor(void) {
    runMotor(180, 0); // Run motor for 180 micro-seconds to close
    SegmentLCD_Write("CLOSING");
    delay(2000);
    SegmentLCD_Write("CLOSED");
```

```c
void runMotor(int duration, int opening) {
    setupPWM();
    // Set the motor direction
    if (opening) {

        GPIO_PinOutSet(MOTOR_INPUT1);
        GPIO_PinOutClear(MOTOR_INPUT2);
    } else {

        GPIO_PinOutClear(MOTOR_INPUT1);
        GPIO_PinOutSet(MOTOR_INPUT2);
    }

    // Run the motor for the specified duration to limit the opening distance
    delay(duration); // Use the adjusted duration for opening/closing

    stopMotor();
}
void stopMotor(void) {



    // Disable motor direction pins
    GPIO_PinOutClear(MOTOR_INPUT1);
    GPIO_PinOutClear(MOTOR_INPUT2);
}
```

הסבר על אתחולים:

אתחולים של ARM:

setupPWM: אתחול של Timer0

```
void setupPWM(void) {
    int presc = 6;
    //freq_hfprerclk = 48000000;
    //freq_pwm=1000;
    CMU_ClockEnable(cmuClock_TIMER0, true);
    GPIO_PinModeSet(MOTOR_EN, gpioModePushPullDrive, 0);
  //TIMER0 Init
    TIMER0->CTRL = (presc<<24);
    TIMER0->CNT = 0x0;  // clear TIMER0 counter value
    TIMER0->TOP = 750;   // Set TOP value, 64 = 2^presc
    TIMER0->CC->CCV =270;  // set compare value
    TIMER0->ROUTE = (3<<16)|0x1;// conect channel to port D pin 1
    TIMER0->CC->CTRL = 3;// config PWM
    TIMER0->CMD = 1; // start timer
}
```

הפעלת פין PD1 במצב PushPullDrive, ואחרי שחישבנו הערכים הנדרשים לאתחול ה-Timer 0. אתחלנו והפעלנו גם את המצב של capture/compare ואתחלנו אותו לפי הערך של ה-duty cycle ובסוף התחלנו ה-Timer.

Init_Device: אתחול של GPIO pins ומסך ה SegmentLCD

```
#define SENSOR_A   gpioPortD,0
#define SENSOR_B   gpioPortD,3
#define SENSOR_C   gpioPortD,2
#define MOTOR_EN   gpioPortD,1
#define MOTOR_INPUT1 gpioPortD,5
#define MOTOR_INPUT2 gpioPortD,6
```

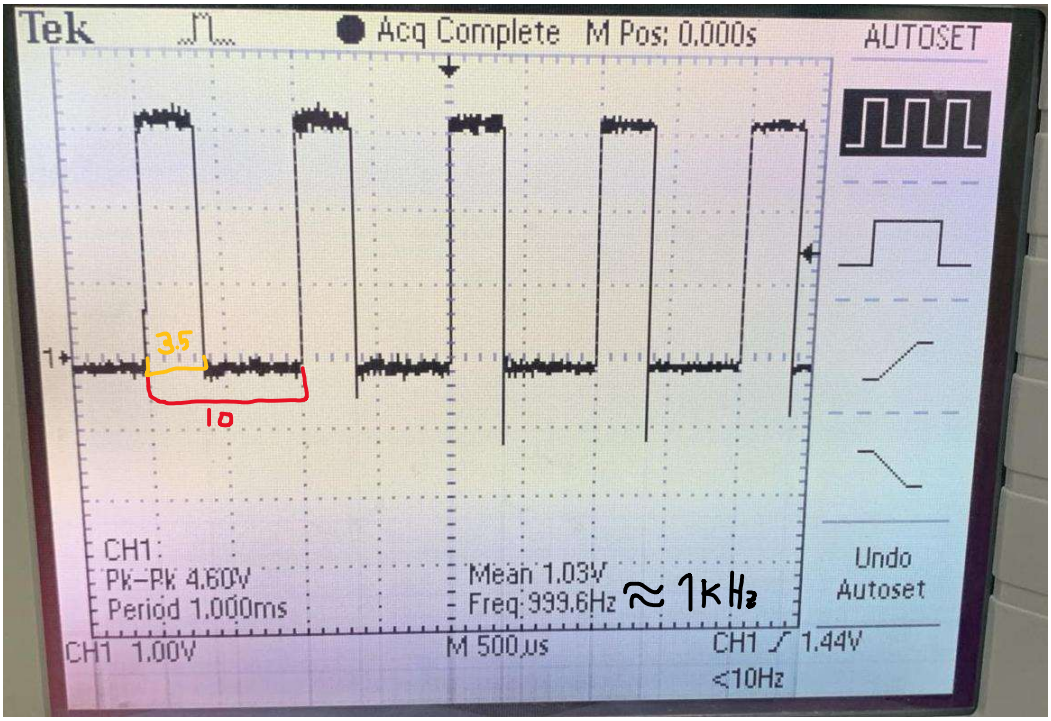
```
void Init_Device(void) {
    SegmentLCD_Init(false);
    CMU_ClockSelectSet(cmuClock_HF, cmuSelect_HFXO);
    CMU_ClockEnable(cmuClock_GPIO, true);

    GPIO_PinModeSet(SENSOR_A, gpioModeInputPull, 1);
    GPIO_PinModeSet(SENSOR_B, gpioModeInputPull, 1);
    GPIO_PinModeSet(SENSOR_C, gpioModeInputPull, 1);
    GPIO_PinModeSet(MOTOR_INPUT1, gpioModePushPull, 0);
    GPIO_PinModeSet(MOTOR_INPUT2, gpioModePushPull, 0);
}
```

הפינים של החישנים במצב input והפינים של ה motor במצב PushPull output

צילומי סקופ של PWM

דרשנו שהתדר של ה-PWM יהיה $1000\ [Hz]$, וה-duty cycle שווה 35%



כמו שרואים, התדר הוא $999.6 \approx 1000[Hz]$ וה-duty cycle: 3.5/10 שזה 35%

תיאור תקלות ודרכי פתרון:

תיאור הבעיה	דרך הפתרון
המנוע היה עובד מהר מאוד ועושה סיבובים למראת שאנחנו צריכים שיסתובב קצת כדי שיהיה לנו שער שנפתח קצת ונסגר	שימוש ב PWM כדי להגדיר Duty cycle מסוים להפחית מהירות המנוע, והגדרת זמן קצר לעבודת המנוע כדי שלא ישלים סיבוב מלא.
לא הצלחנו להציג ולראות אות ה PWM בסקופ	הגדרנו למנוע זמן ארוך לעבוד כדי שיהיה לנו מספיק זמן להציג אות ה PWM בסקופ
לא הצלחנו בהתחלה להפעיל המנוע	חברנו את כל הרכבים לפי datasheet של motor driver